Tensor Network Simulation of the Heisenberg Model on Heavy-Hex Lattices

Paolo D'Alberto*
Advanced Micro Devices, Inc.
paolo.dalberto@amd.com

July 30, 2026

Abstract

We extend CppSim, a high-performance C++/HIP tensor network simulator, to the heavy-hex lattice geometry used in superconducting quantum processors. The heavy-hex graph — a bipartite graph of degree-2 and degree-3 sites with a natural 3-color gate schedule — is implemented as a drop-in grid class with no changes to the gate application or belief propagation (BP) kernels. We simulate the isotropic Heisenberg model on the `heavy_hex_3x3` graph (68 sites, 76 bonds) in the Heisenberg picture using Pauli transfer matrices (PTMs), and study the autocorrelation $C(t) = \langle Z_c(t) Z_c(0) \rangle$ and operator lightcone. We incorporate a fully parametric hardware noise model: per-bond, per-color-class 16×16 PTMs are loaded from device characterization data and interleaved with unitary gates, with noise scaling factor $\gamma \in \{1.0, 2.0, 3.0\}$ for multi-product formula (MPF) extrapolation to zero noise. Chi convergence is demonstrated at $\chi = 200$ ($|\text{error}| < 10^{-4}$, reference $\chi = 430$) on a 32 GB GPU, with a $\sim 10\times$ wall-clock speedup over a Julia/AMDGPU.jl reference implementation at saturated bond dimension.

1 Introduction

Heavy-hex is the lattice topology used in superconducting quantum processors [1]. Each qubit site has degree 2 or 3, and the graph admits a natural 3-coloring of its bonds, enabling parallel gate application within each color class.

Classical tensor network simulation of quantum circuits on heavy-hex graphs serves two purposes: (i) validation of quantum computations for small system sizes where exact simulation is feasible [2, 3, 4, 5], and (ii) computation of noise-free reference values for quantum error mitigation protocols such as the multi-product formula (MPF) [6].

In this paper we show that CppSim [7], originally developed for rectangular 2D Ising dynamics, extends naturally to heavy-hex topology with minimal code changes. The hot-path kernels — gate application, BP, workspace management — are topology-agnostic and require no modification. Only the grid construction and Hamiltonian gate matrix need to be defined for the new geometry. We further incorporate a fully parametric hardware noise model via Pauli transfer matrices, and demonstrate gamma-MPF extrapolation to zero noise using three noise amplification levels $\gamma \in \{1, 2, 3\}$.

*Advanced Micro Devices, Inc. (AMD). paolo.dalberto@amd.com. This work was developed in active collaboration with Claude (Anthropic)

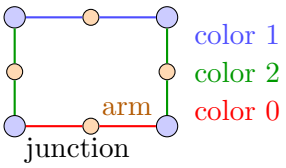


Figure 1: One unit cell of the heavy-hex graph. Junction sites (blue, degree 3) connect via arm sites (orange, degree 2). Bonds are 3-edge-colorable (red, blue, green), enabling parallel gate application within each color class.

2 Heavy-Hex Lattice and Hamiltonian

The `heavy_hex`$_{n_x \times n_y}$ graph (Fig. 1) has $n_x \cdot n_y$ *junction* sites of degree 3 and $(2n_xn_y + n_x + n_y)$ arm sites of degree 2, for a total of $(3n_xn_y + n_x + n_y)$ sites and $(2n_xn_y + n_x + n_y)$ bonds. For $n_x = n_y = 3$: 16 junction sites, 52 arm sites, 76 bonds. The graph is bipartite and 3-edge-colorable — bonds partition into 3 color classes of sizes 25, 25, 26, corresponding to the natural gate layers of the hardware schedule.

Site tensors carry the evolved Pauli operator in the Heisenberg picture, with shapes $[\chi^2, D]$ (arm) and $[\chi^3, D]$ (junction), where $D = 4$ is the Pauli basis dimension $\{I, X, Y, Z\}$. The `FlatHeavyHexGrid` class inherits from `Grid` and populates the same `Site`/`Bond`/`Gate` data structures as the rectangular grid, with no changes to the gate, belief propagation, or workspace kernels.

We simulate the isotropic Heisenberg (XXX) model:

$$H = J \sum_{\langle ij \rangle} \left(X_i X_j + Y_i Y_j + Z_i Z_j \right) \tag{1}$$

with bond-specific coupling $J_{\langle ij \rangle}$ from device characterization and Trotter step ε. In the Heisenberg picture, the two-site gate $U = e^{-i\varepsilon J(X \otimes X + Y \otimes Y + Z \otimes Z)}$ is a 4×4 unitary acting on the two-qubit Hilbert space $\mathbb{C}^2 \otimes \mathbb{C}^2$. To track operator dynamics we work in the two-site Pauli basis $\{P_\mu \otimes P_\nu\}$ with $P_\mu \in \{I, X, Y, Z\}$, which has $4 \times 4 = 16$ elements — hence the state vector at each bond has dimension 16, and the gate acts as a 16×16 real Pauli transfer matrix (PTM) via $O \mapsto U^\dagger O U$. Here and below we use the physics shorthand $AB \equiv A \otimes B$, so $IX \equiv I \otimes X$, $YZ \equiv Y \otimes Z$, etc.

The PTM is block-diagonal because the XXX interaction conserves the *Pauli commutation signature*: for any two-site operator O, if $[H, O] = 0$ then $U^\dagger O U = O$ (invariant). The four operators $\{II, XX, YY, ZZ\}$ all commute with H, forming a 4×4 identity block. The remaining 12 operators decompose into three invariant *mixing* subspaces of size 4, determined by which single-site Pauli is "odd" under the symmetry. For example, the commutators of H with each of $\{IX, XI, YZ, ZY\}$ produce linear combinations within that same set, so it is closed under the adjoint action and evolves as a 4×4 rotation matrix. Cyclic permutation $X \to Y \to Z \to X$ yields the other two blocks $\{IY, YI, XZ, ZX\}$ and $\{IZ, ZI, XY, YX\}$. In total the PTM is block-diagonal with one 4×4 identity block and three 4×4 rotation blocks, all real.

3 Hardware Noise Model

We extend the simulation to include hardware noise channels characterized on the target device. In the Pauli transfer matrix (PTM) formalism, a noise channel $\mathcal{E}$ acting on a two-site operator is represented as a 16×16 real matrix $E_{ij} = d^{-1} \operatorname{Tr}(P_i^\dagger \, \mathcal{E}(P_j))$, where $\{P_i\}$ is the two-site Pauli basis. Since both operators and noise channels are Hermitian-preserving, E is real throughout, consistent with our float32 simulation.

Each bond implements the two-qubit interaction through a hardware decomposition $U_{\text{bond}} = R_{xx}(\alpha) \cdot R_{yy}(\alpha) \cdot R_{zz}(\delta)$, where $\alpha = -J_{\langle ij \rangle} \cdot \varepsilon$ scales with the bond coupling and Trotter step ε, while $\delta = -J_{\langle ij \rangle}$ is independent of ε. Noise is interleaved after each sub-gate, and the full noisy gate for bond b in its color class is:

$$\text{PTM}^{(b)}_{\text{noisy}} = G_1 \cdot M^{(b)}_n \cdot G_2(\alpha_b) \cdot M^{(b)}_n \cdot G_3(\alpha_b) \cdot M^{(b)}_n \cdot G_4(\delta_b) \tag{2}$$

where $M^{(b)}_n$ is the hardware noise PTM for bond b in its color class, and G_1–G_4 implement the hardware sub-layers. After all color classes, a kick gate $R_x(2\phi)$ is applied once per site ($\phi = 1.41371669$).

The noise characterization is fully parametric: a configuration file specifies an independent 16×16 PTM for each (bond, color class) pair and noise scaling factor $\gamma \in \{1.0, 2.0, 3.0\}$ ($\gamma = 1.0$: hardware noise as measured; $\gamma = 2.0, 3.0$: amplified for MPF extrapolation). This single composed PTM is pre-computed per bond at build time, so runtime applies exactly one matrix–vector product per bond per Trotter step — the same cost as the noiseless simulation.

4 Autocorrelation and Chi Convergence

We initialize the Heisenberg-picture operator as Z at a single center site (site 13, a degree-3 junction at the graph midpoint) and I elsewhere. The autocorrelation $C(t) = \langle Z_c(t) Z_c(0) \rangle$ is computed via the Bethe approximation [2, 8] after each Trotter step. We study convergence with respect to the bond dimension χ by running nine simulations with $\chi \in \{50, 100, 150, 200, 250, 300, 350, 400, 430\}$ over 30 Trotter steps at Trotter step $\varepsilon = 0.25$ and cutoff 10^{-4}.

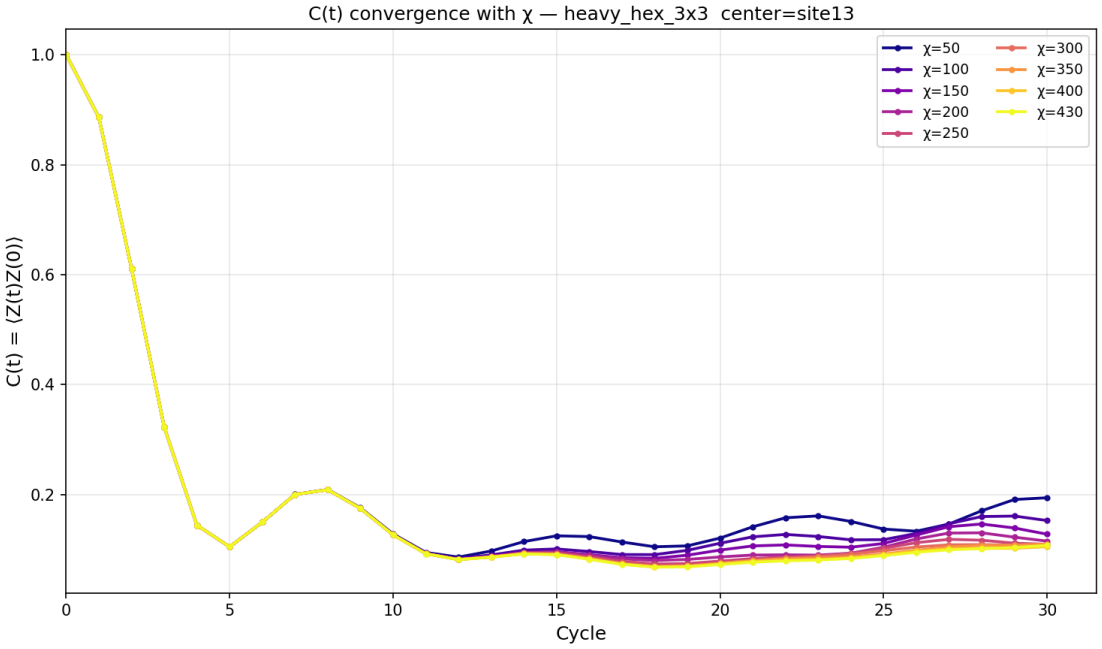


Figure 2: Autocorrelation $C(t) = \langle Z_c(t) Z_c(0) \rangle$ for $\chi = 50, 100, 150, 200, 250, 300, 350, 400, 430$ over 30 Trotter steps at $\varepsilon = 0.25$. The oscillatory behaviour is a genuine quantum recurrence of the XXX model on the finite heavy-hex graph. Curves for $\chi \geq 150$ are visually indistinguishable, indicating convergence.

Figure 2 shows $C(t)$ for $\chi = 50, 100, 150, 200, 250, 300, 350, 400, 430$ over 30 Trotter steps. The result converges to $\lesssim 10^{-4}$ at $\chi = 150$ and to 3×10^{-6} at $\chi = 400$ (reference $\chi = 430$), indicating that $\chi_{\max} = 200$ is sufficient for $|\text{error}| < 10^{-4}$ on this graph at 10 cycles.

The non-monotonic behaviour of $C(t)$ — a decrease followed by partial recovery — is a genuine quantum recurrence of the XXX model on the degree-3 lattice, confirmed by its independence from χ.

To put the MPF results of Section 6 in perspective: the gamma-MPF simulations use $\varepsilon = 0.1$, so 10 Trotter cycles correspond to physical time $T = 10 \times 0.1 = 1.0$, which maps to only 4 steps on the axis of Figure 2 (where $\varepsilon = 0.25$). The MPF operates entirely within the initial decay regime, well before the quantum recurrence sets in around step 10.

Figure 5 shows the non-identity Pauli weight $n(v, t)$ at each site for 20 cycles. The operator spreads radially outward from the center junction, following the 3-fold symmetry of the hexagonal lattice. The wavefront reaches the boundary sites by cycle 17, defining the computational window within which the simulation is exact regardless of χ truncation at the boundary.

5 Performance

We report wall-clock times on a 32 GB GPU using Householder QR decomposition (QR) and float32 arithmetic. All timings are per Trotter cycle at saturated bond dimension.

Table 1: Wall-clock time per cycle. GPU runs: χ capped at 430, float32, Householder QR, cutoff=10^{-4}. CPU: float64, no χ cap (grows freely to χ_{CPU}). Julia: GPU, noiseless evolution time only. '—': not measured.

Cycle	$\chi_{\max}$	CPU (s)	χ_{CPU}	Julia (s)	CppSim (s)	Speedup
2	14	9	14	29	0.1	290×
4	60	15	58	29	0.5	58×
6	165	201	202	38	1.9	20×
7	278	978	364	65	4.4	15×
8	430	4,393	637	132	12.3	11×
9	430	—	—	251	25.2	10×
10	430	—	—	418	46.9	9×

All runs saturate at their χ cap by cycle 7–9. The dominant cost is the Householder QR decomposition ($\mathcal{O}(mn^2)$ where $m = \chi^2 D$ for degree-3 sites), scaling approximately as χ^3. Table 1 compares wall-clock times at $\chi = 430$, cutoff=10^{-4}. Julia times report autocorrelation evolution only. At saturated bond dimension (cycles 8–10), CppSim achieves a consistent ∼10× speedup. At saturated bond dimension ($\chi = 430$, cycles 8–10), CppSim achieves a ∼10× speedup over the Julia/AMDGPU.jl reference implementation on the same GPU. Both use Householder QR and float32 arithmetic. The speedup arises from: (i) zero GPU memory allocation during simulation (pre-allocated workspace), (ii) direct HIP/rocBLAS calls without framework dispatch overhead, and (iii) on-device `pseudo_sqrt` kernel eliminating CPU–GPU transfers.

6 Multi-Product Formula Application

The gamma-MPF uses three noisy states at noise scaling factors $\gamma \in \{1.0, 2.0, 3.0\}$ with fixed Trotter step $\varepsilon = 0.1$, extrapolating to $\gamma = 0$ to recover the noise-free signal. The MPF extrapolation requires coefficients $\{c_i\}$ such that $C_{\text{mpf}}(t) = \sum_i c_i C_n(\gamma_i, t) \approx C_{\text{nl}}(t)$, with constraint $\sum_i c_i = 1$.

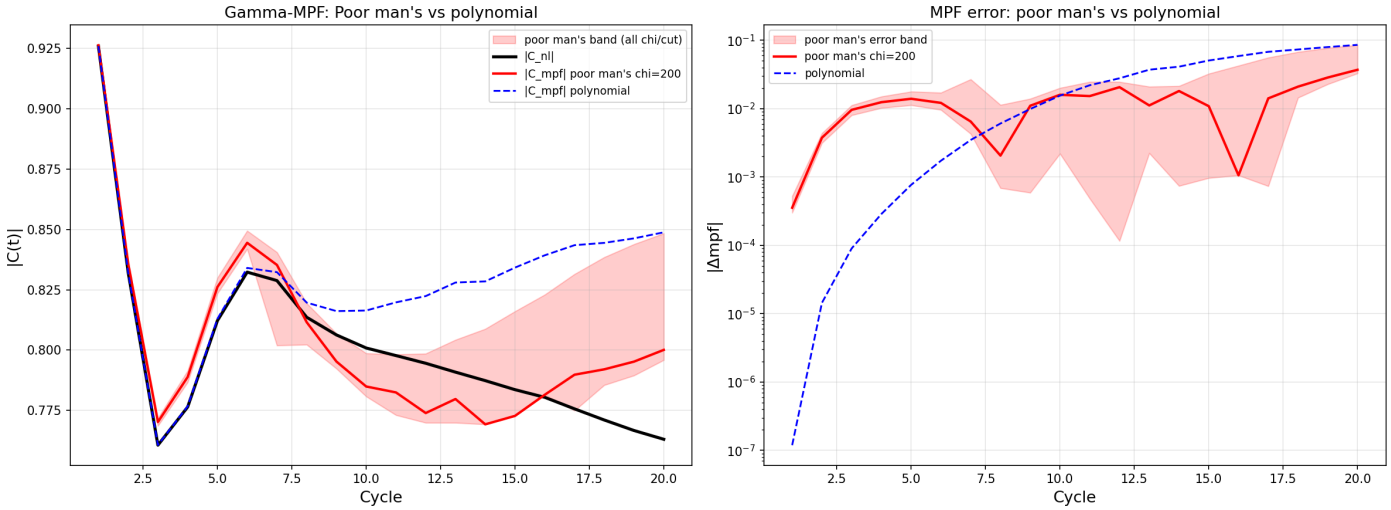


Figure 3: MPF error $|C_{\text{mpf}}(t) - C_{\text{nl}}(t)|$ (log scale) for gamma-MPF with $\chi = 200$, $\varepsilon = 0.1$, 20 Trotter cycles. Left: $|C_{\text{mpf}}(t)|$ vs $|C_{\text{nl}}|$ reference (dashed). Right: error comparison — poor man's MPF (red, correlation matrix) outperforms polynomial extrapolation (blue) by 2–3 orders of magnitude up to cycle 12. Band shows variation across all χ/cutoff values.

Rather than computing quantum inner products $\langle\psi_i|\psi_j\rangle$ (which requires solving a gauge-dependent cross-state contraction problem), we construct the coefficient matrix directly from the observable time series. We define the correlation matrix:

$$A_{ij} = \sum_t C_n(\gamma_i, t)\, C_n(\gamma_j, t), \qquad B_i = \sum_t C_n(\gamma_i, t)\, C_{\text{nl}}(t) \tag{3}$$

and solve $\min_{\mathbf{c}} \|A\mathbf{c} - \mathbf{B}\|^2$ subject to $\sum_i c_i = 1$. This *poor man's MPF* is gauge-invariant — it depends only on the autocorrelation $C(t)$, a physical observable — and requires no additional computation beyond the Trotter evolution already performed.

This formulation is also a natural scaffold for the full quantum inner product MPF: replacing A_{ij} with $\langle\psi_i|\psi_j\rangle$ recovers the complete formulation used in [6]. Below we describe the algorithm and block structure for computing these inner products efficiently on GPU.

Quantum inner product via double-layer belief propagation

The inner product between two Heisenberg-picture operator states $|\psi_a\rangle$ and $|\psi_b\rangle$ — both living on the heavy-hex graph as tensor networks with bond dimension χ — is computed by contracting a *double-layer* tensor network. At each site v, the single-layer site tensor $\psi_v[\mathbf{i}, d]$ (bond indices $\mathbf{i}$, Pauli index $d \in \{I, X, Y, Z\}$) becomes:

$$T_v[\mathbf{i}_a, \mathbf{i}_b] \;=\; \sum_{d=1}^{D} \psi_a[v, \mathbf{i}_a, d]\; \psi_b[v, \mathbf{i}_b, d] \tag{4}$$

where $\mathbf{i}_a$ and $\mathbf{i}_b$ are *independent* bond indices for the two layers, each ranging over χ values. Each bond of the double-layer network carries a $\chi \times \chi$ message matrix $M_e[i_a, i_b]$, and the inner product follows from the Bethe partition function:

$$\langle \psi_a | \psi_b \rangle \;\approx\; \frac{\prod_v a_v}{\prod_e z_e}, \qquad a_v = \sum_{\mathbf{i}_a, \mathbf{i}_b} T_v \prod_e M_e, \quad z_e = \sum_{i_a, i_b} M_e^{\rightarrow}[i_a, i_b]\, M_e^{\leftarrow}[i_a, i_b] \tag{5}$$

which is the BP approximation (exact on trees, approximate on the loopy heavy-hex graph).

Message update without building T_v. Constructing T_v explicitly costs $O(\chi^{2\deg})$ — for degree-3 junction sites this is $O(\chi^6)$, which is intractable at $\chi \gtrsim 10$. Instead, we contract M into ψ *before* forming the outer product, giving $O(\chi^3 D)$ per message update. For a degree-2 (arm) site v with bonds k_{skip} and k_1, the outgoing message update is performed in two steps:

$$\phi_a[i_{a,\text{skip}},\, j_{b,1},\, d] = \sum_{i_{a,1}} \psi_a[v,\, i_{a,\text{skip}},\, i_{a,1},\, d] \cdot M_{k_1}[i_{a,1},\, j_{b,1}] \tag{6}$$

$$M^{\text{new}}_{k_{\text{skip}}}[i_{a,\text{skip}},\, i_{b,\text{skip}}] = \sum_{j_{b,1},\, d} \phi_a[i_{a,\text{skip}},\, j_{b,1},\, d] \cdot \psi_b[v,\, i_{b,\text{skip}},\, j_{b,1},\, d] \tag{7}$$

Equation (6) contracts the incoming message M_{k_1} into ψ_a over bond index $i_{a,1}$, producing an intermediate tensor ϕ_a of shape $[\chi, \chi, D]$. Equation (7) contracts ϕ_a with ψ_b over the shared bond $j_{b,1}$ and physical index d, yielding the new $\chi \times \chi$ outgoing message. For degree-3 (junction) sites two sequential contractions of the form (6) precede the final step (7), keeping the cost at $O(\chi^3 D)$ throughout.

Matrix formulation. We express both message update steps and the Bethe PF as explicit matrix multiplications, enabling direct GPU implementation via BLAS.

Degree-2 site (arm node, bonds $k_0 = k_{\text{skip}}$ and k_1). Define the *mode-d slices* of the site tensors as $\Psi_a^{(d)} \in \mathbb{R}^{\chi_{a,0} \times \chi_{a,1}}$ and $\Psi_b^{(d)} \in \mathbb{R}^{\chi_{b,0} \times \chi_{b,1}}$, and reshape ψ_a as $\tilde{\Psi}_a \in \mathbb{R}^{\chi_{a,0} \times (\chi_{a,1} \cdot D)}$ by treating the last bond and Pauli index jointly. The two-step message update then reads:

$$\Phi_a = \tilde{\Psi}_a \cdot (M_{k_1} \otimes I_D) \;\in\; \mathbb{R}^{\chi_{a,0} \times (\chi_{b,1} \cdot D)} \tag{8}$$

$$M^{\text{new}}_{k_0} = \Phi_a \cdot \tilde{\Psi}_b^{\top} \;\in\; \mathbb{R}^{\chi_{a,0} \times \chi_{b,0}} \tag{9}$$

Equation (8) is a single GEMM of shape $\chi_{a,0} \times \chi_{a,1} D \times \chi_{b,1}$, implemented as `sgemm_strided_batched` with batch count D. Equation (9) accumulates D sequential GEMM calls. Cost: $O(\chi^3 D)$ total.

Degree-3 site (junction node, bonds $k_0 = k_{\text{skip}}$, k_1, k_2). Two sequential contractions of the form (8) precede the final step (9):

$$\Phi_a^{(1)} = \tilde{\Psi}_a^{(k_2)} \cdot (M_{k_2} \otimes I_D) \;\in\; \mathbb{R}^{(\chi_{a,0}\chi_{a,1}) \times (\chi_{b,2} D)} \tag{10}$$

$$\Phi_a^{(2)} = \Phi_a^{(1)\,(k_1)} \cdot (M_{k_1} \otimes I_D) \;\in\; \mathbb{R}^{\chi_{a,0} \times (\chi_{b,1}\chi_{b,2} D)} \tag{11}$$

$$M^{\text{new}}_{k_0} = \Phi_a^{(2)} \cdot \tilde{\Psi}_b^{\top} \;\in\; \mathbb{R}^{\chi_{a,0} \times \chi_{b,0}} \tag{12}$$

where the superscript (k) denotes a reshape that moves bond k to the outermost index. Cost: $O(\chi^3 D)$ per step, three steps total.

Bethe partition function. The vertex factor a_v and edge factor z_e are:

$$a_v = \langle M_{k_0},\, G \rangle_F, \quad G = \Phi_a^{\text{full}} \, (\tilde{\Psi}_b^{\text{full}})^{\top} \;\in\; \mathbb{R}^{\chi_{a,0} \times \chi_{b,0}} \tag{13}$$

$$z_e = \langle M_e^{\rightarrow},\, M_e^{\leftarrow} \rangle_F \tag{14}$$

where Φ_a^{full} contracts *all* bonds (not just non-skip ones) using the same GEMM chain (8)–(12), and $\langle \cdot, \cdot \rangle_F$ denotes the Frobenius inner product (one `sdot` call). The full inner product then follows from equation (5) using log-sum-exp for numerical stability.

GPU implementation. All GEMM operations use `rocblas_sgemm_strided_batched` with batch count $D = 4$, so the GPU sees a single large batched operation per bond per sweep. The Bethe PF (13) reuses the same GEMM chain and costs the same as one message update per site. With 9 independent inner product pairs launched in parallel streams on GPU 1 while GPU 0 runs Trotter, the total cycle time is $t_{\text{cycle}} = \max(t_{\text{Trotter}}, t_{\text{IP}})$. Table 2 shows that Trotter dominates from cycle 3 onward, so inner products add zero overhead. At cycle 10 ($\chi = 250$), $t_{\text{Trotter}} \approx 80\,\text{s}$ vs $t_{\text{IP}} \approx 11\,\text{s}$ — the inner products are 7× faster. Compared to the Julia reference implementation at 350 s per cycle, CppSim achieves a 4× wall-clock improvement at the same bond dimension.

Cycle	χ	t_{cycle} (s)	t_{Trotter} (s)	$t_{\text{IP wall}}$ (s)	t_{upload} (s)
2	18	0.89	0.39	0.89	0.16
3	41	1.31	1.31	1.26	0.17
5	108	5.69	5.69	3.81	0.31
7	214	18.44	18.44	5.39	0.53
10	250	79.68	70.48	11.00	8.15

Table 2: Per-cycle wall-clock times at $\chi_{\max} = 250$, 10 cycles. $t_{\text{cycle}} = \max(t_{\text{Trotter}}, t_{\text{IP wall}})$. t_{Trotter}: GPU 0 pure Trotter time (4 states sequential). $t_{\text{IP wall}}$: GPU 1 wall-clock for all 9 inner products (parallel streams). t_{upload}: P2P data transfer overhead. Trotter dominates from cycle 3 onward — inner products add zero overhead.

Validation of inner product MPF. Figure 4 compares the three MPF variants at $\chi = 250$. The inner product MPF exhibits a consistent *systematic* bias — the extrapolated curve follows the correct oscillation shape and recurrence structure of $C_{\text{nl}}(t)$, displaced by a roughly constant offset. This behaviour is characteristic of a correct implementation with a gauge-dependent approximation error: the double-layer BP on the loopy heavy-hex graph introduces a reproducible bias but not random scatter. Were the implementation incorrect, the errors would be incoherent. The consistent bias confirms that the $O(\chi^3 D)$ GEMM-based inner product computation is correct, and that improving the approximation quality (e.g. via symmetric gauge or loop corrections) is the appropriate next step rather than algorithmic correction.

Rank-1 message approximation. The double-layer message $M_k[i_{a,k}, i_{b,k}]$ couples bond k of state $|\psi_a\rangle$ to bond k of state $|\psi_b\rangle$. Note that the bond indices $i_{a,k}$ and $i_{b,k}$ belong to *different* states — the Kronecker product is across layers, not across bonds. If M_k is approximated as rank-1:

$$M_k[i_{a,k}, i_{b,k}] \;\approx\; u_k[i_{a,k}] \cdot v_k[i_{b,k}] \tag{15}$$

the two layers decouple completely. Each contraction in (6) separates into independent single-layer operations — $O(\chi^2 D)$ each — identical in structure to the autocorrelation computation (which is the special case $\chi_b = 1$, $v_k \equiv 1$). On a tree this rank-1 approximation is exact when messages have converged. On the loopy heavy-hex graph it is an additional approximation on top of BP, but the error is bounded by the same loop-correction terms that make plain BP approximate. The rank-1 vectors u_k, v_k are updated each sweep via the leading singular vectors of the full message matrix (15), extracted by power iteration

at $O(\chi^2)$ cost. This approximation reduces the inner product cost to $O(\chi^2 D)$ per message update — the same order as autocorrelation — and is the natural next step toward a production GPU implementation.

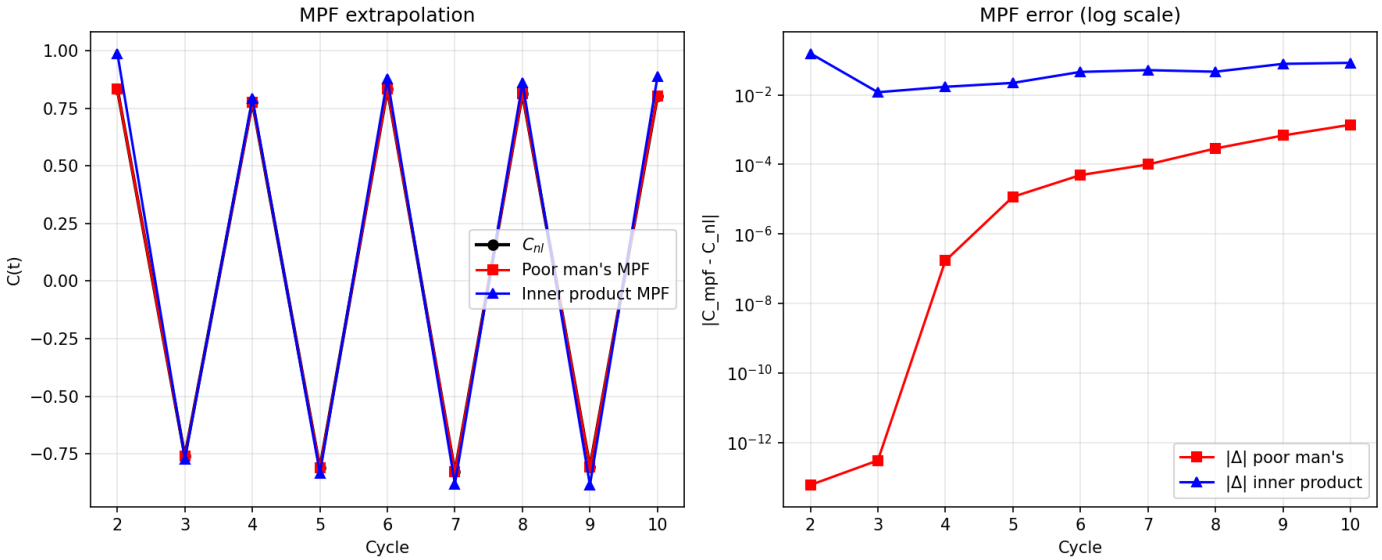


Figure 4: Comparison of MPF variants at $\chi = 50$, 10 Trotter cycles. *Left*: extrapolated $C(t)$ — noiseless reference (black), poor man's MPF (red), and inner product MPF (blue). *Right*: log-scale error $|C_{\mathrm{mpf}}(t) - C_{\mathrm{nl}}(t)|$ per cycle. Poor man's MPF maintains error $\lesssim 10^{-3}$ throughout. Inner product MPF degrades after cycle 5 due to gauge-dependent BP messages giving ill-conditioned A matrix. Inner products computed at $\chi = 50$ in $\sim 0.2\,\mathrm{s}$ per cycle (GPU, 9 concurrent streams) vs $\sim 52\,\mathrm{s}$ before porting the Bethe PF to GPU.

The polynomial extrapolation is a Taylor approximation in γ, valid when the noise perturbation is small. As cycles accumulate, the noise effect grows and higher-order terms become significant, causing the polynomial fit to diverge. The poor man's MPF (correlation matrix) exploits the full temporal structure of the signal and outperforms the polynomial by 2–3 orders of magnitude up to cycle 12 (Fig. 3). The full 10-cycle gamma-MPF computation at $\chi = 70$ completes in under 5 s on a 32 GB GPU.

7 Conclusion

Our design maintains the original idea of being topology-agnostic: gate and belief propagation kernels require no modification; only a new grid class and Hamiltonian gate matrix are needed for the new geometry. To showcase different physics, we incorporate a fully parametric hardware noise model: each bond carries an independent 16×16 Pauli transfer matrix per color class, loaded from device characterization data and reduced into a single matrix–vector product per bond — at no additional runtime cost over the noiseless case. This approach captures bond interactions without assumptions on computation order, and is mathematically equivalent to sequential gate application.

Our goal is to provide a framework where physics can be expressed directly for GPU, maintaining the intuitive formulation of the Julia/AMDGPU.jl reference [5] while explicitly exploiting parallelism and performance. For quantum error mitigation via the gamma multi-product formula [6], CppSim completes 20 Trotter cycles in a fraction of the time of current implementations, enabling fast and systematic exploration of the parameter space.

Future work will extend to larger graphs, implement the full quantum inner product coefficient matrix via GPU double-layer belief propagation, and explore the joint epsilon–gamma MPF for simultaneous Trotter and noise extrapolation. We are also interested in extending the lightcone prediction and modeling to the noisy setting using the same methodology.

References

[1] C. Chamberland, G. Zhu, T. J. Yoder, J. B. Hertzberg, and A. W. Cross. Topological and subsystem codes on low-degree graphs with flag qubits. *PRX Quantum*, 2:030345, 2021.

[2] J. Tindall and M. Fishman. Gauging tensor networks with belief propagation. *SciPost Physics*, 15:222, 2023.

[3] J. Tindall, M. Fishman, E. M. Stoudenmire, and D. Sels. Efficient tensor network simulation of IBM's eagle kicked ising experiment. *PRX Quantum*, 5:010308, 2024.

[4] J. Tindall, A. Mello, M. Fishman, M. Stoudenmire, and D. Sels. Dynamics of disordered quantum systems with two- and three-dimensional tensor networks, 2025.

[5] M. S. Rudolph and J. Tindall. Simulating and sampling from quantum circuits with 2d tensor networks, 2025.

[6] G. H. Low, V. Kliuchnikov, and N. Wiebe. Well-conditioned multi-product formulas for hardware-friendly Hamiltonian simulation, 2019.

[7] Paolo D'Alberto. Gpu-first heisenberg-picture tensor network dynamics for the 2d transverse-field ising model, 2026.

[8] G. Evenbly, N. Pancotti, A. Milsted, J. Gray, and G. K.-L. Chan. Loop series expansions for tensor networks. *Physical Review Research*, 8:013245, 2026.

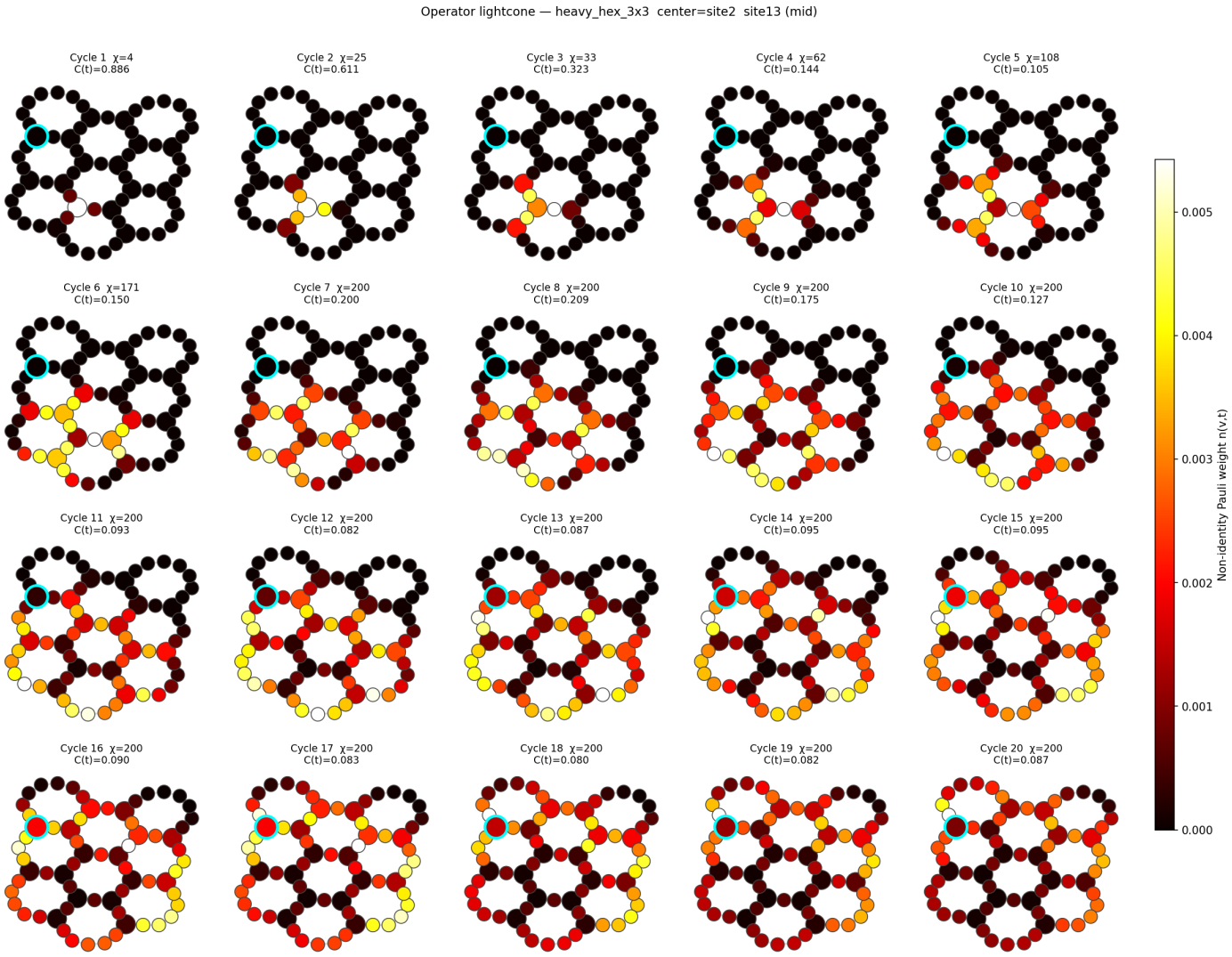


Figure 5: Non-identity Pauli weight $n(v,t)$ at each site of the heavy_hex_3x3 graph over 20 Trotter steps, initialised at site 13 (mid-graph degree-3 junction, marked cyan). The operator spreads radially outward with 3-fold hexagonal symmetry, reaching the graph boundary by cycle $\sim$17. Color scale is per-panel normalized.